

Prática: construir o TaskWeather

Criar uma aplicação web simples a partir do template, com planejamento, prompts estruturados, skills, validação local e versionamento.

2H BASE → 3 INCREMENTOS → TEMPLATE → PLANO → ENTREGA

Escopo da aplicação

O que entra, o que é opcional e o que fica de fora do TaskWeather.

ENTRA AGORA

Obrigatório

- Entrada por e-mail sem senha, sessão local, logout
- Dados separados por usuário
- Criar, listar e concluir tarefas
- Persistência local
- Validação local e documentação mínima

TALVEZ

Opcional (se houver tempo)

- Clima atual, com loading e erro
- Editar e excluir tarefa
- Filtrar tarefas
- Testes adicionais
- Melhoria visual
- PR revisado por outro grupo

NÃO ENTRA

Fora do escopo

- Backend e banco de dados
- Autenticação real, senha e cadastro
- Recuperação de senha / dados sensíveis
- Geolocalização automática
- Previsão avançada e publicação em produção

Agenda-base de 120 minutos

Introdução curta e execução prática guiada, em checkpoints comuns.

Etapa	Atividade	Duração
1	Abertura e regras	8 min
2	Criar projeto e executar template	10 min
3	Planejar — reusando o plano do Dia 2	8 min
4	Entrada por e-mail	20 min
5	Lista de tarefas	25 min
6	Separação de dados por usuário	15 min
7	Validar, revisar e documentar	20 min
8	Demonstração cruzada e retrospectiva	14 min

Resultado do workshop

Aplicar o processo, não criar o sistema mais completo. Não pedir a aplicação inteira de uma vez.

SIMPLES, COMPREENSÍVEL, EXECUTÁVEL

A aplicação final

- Tela de entrada por e-mail
- Lista de tarefas
- Cartão de clima

TEMPLATE → PLANO → INCREMENTOS → TESTES → REVISÃO

O fluxo

- Começar pelo template
- Orientar o agente
- Trabalhar em incrementos
- Validar cada etapa
- Registrar o resultado

Criar e executar o projeto

Confirmar que o projeto inicia e que o agente leu as regras — antes de construir qualquer coisa.

- Usar o template
- Nomear o repositório
- Abrir no agente
- Instalar dependências
- Executar a aplicação
- Ler as instruções

COMANDOS + PROMPT DE APOIO

```
npm install  
npm run dev
```

```
# Leia README, AGENTS,  
# CLAUDE e architecture.  
# Não altere arquivos.  
# Resuma: objetivo, estrutura,  
# comandos e regras.
```

Planejar antes de implementar

Use a skill plan-feature. Não altere arquivos.

PROMPT (RESUMO)

Use plan-feature para o TaskWeather.

Objetivo: entrada por e-mail, tarefas e clima.

Escopo: sessão local, dados por usuário, criar/listar/concluir tarefas, persistência, clima por cidade, loading/erro.

Fora do escopo: backend, auth real, senha, cadastro, editar/excluir, geoloc.

Restrições: não instalar deps; seguir AGENTS.md; por features.

Saída: requisitos, suposições, proposta, tarefas, riscos, critérios.

CHECKPOINT: 3 INCREMENTOS

Verificar no plano

- Respeita o não escopo?
- Cada funcionalidade está separada?
- Existem critérios observáveis?
- O agente tentou incluir bibliotecas?
- A ordem permite validar um incremento por vez?

Entrada por e-mail

Implementar apenas o incremento de identificação aprovado no plano.

Critérios de aceite

- E-mail válido acessa; vazio ou inválido mostra mensagem
- A tela NÃO pede senha
- Atualizar a página mantém a sessão
- E-mail normalizado: minúsculas, sem espaços
- Logout encerra sem apagar dados; nenhuma auth real

ENTRADA SEM SENHA · INFORME APENAS O E-MAIL

demo@empresa.com

⚠ *Identifica, não autentica.*

ISOLAR A FEATURE

Restrições e processo

- Não instalar deps; não fazer tarefas nem clima
- Não criar senha, cadastro nem recuperação de senha
- Feature isolada em src/features/auth
- Armazenamento por serviço, não no componente
- Informe arquivos → implemente → teste → valide
- Commit: feat: adiciona identificação por e-mail

Lista de tarefas

A funcionalidade central — apenas o básico. Commit: feat: adiciona criação e conclusão de tarefas.

FORA: EDITAR, EXCLUIR, CATEGORIAS, DATAS

Escopo

- Criar tarefa com título
- Listar tarefas
- Concluir / reabrir
- Persistir após recarregar
- Guardar separado por usuário, com o e-mail na chave
- Mostrar estado vazio

COMO VERIFICAR NA TELA

Critérios e restrições

- Título vazio não cria tarefa
- Tarefa válida aparece imediatamente
- Estado permanece após recarregar; identificação intacta
- As tarefas de um e-mail NÃO aparecem para outro
- Feature em src/features/todos; storage encapsulado
- Não instalar deps; não alterar a identificação

Widget meteorológico

Uma integração externa pequena e controlada. Funciona no navegador e não exige segredo.

Cidade → **Coordenadas** → **Clima atual** → **Exibição**

A INTERFACE NÃO PODE TRAVAR

Estados visuais

- Inicial
- Carregando
- Sucesso
- Erro

Escopo e critérios

- Campo de cidade, ação de busca, nova busca
- Exibir cidade, temperatura e condição atual
- Cidade válida exibe clima; falha mostra mensagem
- Serviço indicado em docs; sem segredo no projeto
- Chamadas externas em src/features/weather

Validar e revisar

Comprovar e registrar a entrega. Use a skill review-changes; não altere arquivos inicialmente.

VALIDAÇÕES

```
npm run test  
npm run lint  
npm run typecheck  
npm run build
```

PR — descrição mínima

- Objetivo, funcionalidades, como testar, limitações, evidências

PRIORIDADE + CORREÇÕES MÍNIMAS

review-changes verifica

- Critérios de aceite e alterações fora do escopo
- Dependências, segredos ou credenciais inadequadas
- Separação entre auth, todos e weather
- Acesso direto a storage/APIs em componentes
- Testes ausentes, acessibilidade, docs e código morto

Demonstração e retrospectiva

Selecionar um ou dois grupos para mostrar aplicação, features, um commit e os testes.

1

O que o agente fez bem?

- Onde ele acelerou o trabalho.

2

Onde o processo evitou um problema?

- Escopo, plano, validação.

3

O que melhorar no template?

- A melhoria que deve entrar.

Houve tentativa de expandir escopo? O plano foi útil? Qual instrução precisou ser reforçada?

Checkpoints e critérios de sucesso

Todos os grupos param nos mesmos pontos. O aprendizado prioritário é o processo.

Checkpoints

- 0 · Ambiente: projeto inicia, sem alterações
- 1 · Plano: 3 incrementos e critérios
- 2 · E-mail: válido entra, inválido recusa, sessão, logout
- 3 · Tarefas: criar, concluir, persistir, separadas por usuário
- 4 · Clima: loading, sucesso, erro
- 5 · Entrega: validações, docs, PR

CRITÉRIOS DE SUCESSO

Concluído quando o grupo

- Usou o template e planejou antes de implementar
- Trabalhou em incrementos e respeitou o não escopo
- Executou validações e revisou alterações
- Registrou commits e documentou execução/testes
- Consegue explicar a estrutura do projeto

O artefato é o processo

- Do template ao PR, em incrementos validados
- Escopo, critérios e não escopo explícitos em cada etapa
- Capacidade de repetir isto em outros projetos